

Retrospectiva del Proyecto Slot Machine

Desarrollo Orientado por Objetos [DOPO-POOB] – Ciclo 1

1. ¿Cuáles fueron los mini-ciclos definidos? Justifiquenlos.

Mini-ciclo 1: Preparación del proyecto y reutilización de shapes. Se creó el proyecto slotMachine en BlueJ y se incorporaron las clases Circle, Rectangle, Triangle y Canvas del proyecto shapes. Este mini-ciclo permitió establecer la base gráfica del simulador y cumplir con el requisito de reutilización de componentes.

Mini-ciclo 2: Creación de la máquina tragamonedas. Se construyó la clase principal SlotMachine y se creó su representación visual utilizando un Rectangle. También se implementaron las operaciones básicas relacionadas con la visibilidad de la máquina.

Mini-ciclo 3: Administración de ruedas. Se creó la clase Wheel y se implementaron las operaciones para adicionar, eliminar y organizar las ruedas dentro de la máquina. Para ello se utilizaron métodos auxiliares privados que permiten ajustar posiciones y reorganizar gráficamente las ruedas.

Mini-ciclo 4: Administración de símbolos. Se inició la implementación de la colección de símbolos, permitiendo agregar símbolos sin repetir colores, eliminarlos y consultar los símbolos almacenados.

Mini-ciclo 5: Visibilidad del simulador. Se implementaron los métodos makeVisible() y makeInvisible() para controlar la representación visual de la máquina y de sus ruedas.

Mini-ciclo 6: Diseño y documentación. Se elaboró el diagrama de clases y los diagramas de secuencia correspondientes en Astah, buscando mantener coherencia entre el diseño y el código desarrollado.

Justificación general. La división en mini-ciclos permitió abordar los requisitos de forma incremental, probar cada funcionalidad antes de continuar con la siguiente y facilitar la detección y corrección de errores.

2. ¿Cuál es el estado actual del proyecto en términos de mini-ciclos? ¿Por qué?

Actualmente, el proyecto se encuentra parcialmente terminado. Se implementaron o avanzaron los mini-ciclos relacionados con la creación de la máquina, la administración de ruedas, la administración básica de símbolos, la visibilidad del simulador y parte del diseño en Astah.

Entre las funcionalidades desarrolladas se encuentran:

Creación de la máquina tragamonedas.

Adición de ruedas.

Eliminación de ruedas.

Organización de las ruedas según su posición.

Adición de símbolos.

Eliminación de símbolos.

Consulta de los símbolos almacenados.

Control de la visibilidad del simulador.

Sin embargo, permanecen pendientes o incompletas operaciones como *placeSymbol()*, *spin()*, *distinctSymbols()*, *configuration()*, *isJackpot()*, *exit()* y *ok()*. Por esta razón, existe una estructura funcional del simulador, pero todavía faltan requisitos por completar para cerrar completamente el primer ciclo.

3. ¿Cuál fue el tiempo total invertido por cada uno de ustedes? (Horas/Hombre)

Rodríguez: 7 horas.

Vargas: 8 horas.

Este tiempo incluye actividades de planificación, programación en BlueJ, pruebas, corrección de errores, elaboración de diagramas en Astah y organización del repositorio en GitHub.

4. ¿Cuál consideran fue el mayor logro? ¿Por qué?

El mayor logro fue construir una estructura extensible para representar la máquina tragamonedas y administrar dinámicamente sus ruedas. En lugar de definir una cantidad fija de ruedas, se utilizó una colección `ArrayList<Wheel>`, lo que permite agregarlas y eliminarlas durante la ejecución. Además, se implementó el método `arrangeWheels()` para reorganizar automáticamente sus posiciones.

Este logro fue importante porque facilita la incorporación de nuevas funcionalidades sin modificar por completo la estructura inicial del proyecto y responde al requisito de extensibilidad planteado desde el comienzo.

5. ¿Cuál consideran que fue el mayor problema técnico? ¿Qué hicieron para resolverlo?

Uno de los principales problemas técnicos fue organizar correctamente la representación gráfica de las ruedas. Inicialmente, varias ruedas podían quedar ubicadas en la misma posición visual, haciendo que pareciera que solo existía una. Para resolverlo, se agregó un atributo de posición a la clase `Wheel` y se creó el método `setPosition(int newPosition)`. Posteriormente, se implementó `arrangeWheels()` en `SlotMachine` para recorrer la colección de ruedas y actualizar su posición después de agregar o eliminar una rueda.

Otra dificultad importante fue mantener sincronizados el código y los diagramas de Astah. A medida que se agregaban atributos y métodos al programa, también era necesario actualizar el diseño UML. Para resolverlo, se revisó el código implementado y se ajustaron los diagramas para que representaran el estado real del proyecto.

También se presentaron dificultades con GitHub y la organización del repositorio. Fue necesario aprender a separar correctamente los productos de la entrega, subir los archivos del código y de Astah, organizar las ramas del repositorio y mantener una estructura comprensible. Esto tomó más tiempo del previsto y se resolvió revisando paso a paso el proceso de carga, organización y actualización de los archivos.

6. ¿Qué hicieron bien como equipo? ¿Qué se comprometen a hacer para mejorar los resultados?

Como equipo, se realizó una división progresiva del problema, evitando intentar desarrollar todos los requisitos al mismo tiempo. También se realizaron pruebas después de implementar funcionalidades importantes y se utilizó GitHub para organizar los diferentes productos de la entrega. La comunicación permitió identificar errores y tomar decisiones sobre cómo continuar el desarrollo.

Como compromiso de mejora, se propone planificar con mayor precisión los mini-ciclos antes de comenzar a programar, mantener el diseño de Astah actualizado desde el momento en que se modifica el código y realizar commits más frecuentes para registrar mejor la evolución del proyecto.

Además, se considera necesario comenzar el proyecto con mayor anticipación. No se revisó a tiempo toda la información disponible en la plataforma, lo que redujo el tiempo disponible para completar la entrega. Para próximos ciclos, se revisarán los requisitos y materiales desde el inicio, se distribuirá mejor el trabajo y se reservará tiempo suficiente para pruebas y correcciones finales.

7. Considerando las prácticas XP incluidas en los laboratorios, ¿cuál fue la más útil? ¿Por qué?

La práctica XP más útil fue el desarrollo incremental mediante pequeños ciclos de implementación y prueba. Esta práctica permitió construir primero una funcionalidad pequeña, comprobar su funcionamiento y luego continuar con la siguiente. Por ejemplo, antes de administrar varias ruedas, se comprobó que un objeto Wheel pudiera representarse gráficamente utilizando los componentes de shapes.

Este enfoque ayudó a identificar errores con mayor rapidez y evitó acumular múltiples problemas antes de realizar las pruebas.

8. ¿Qué referencias usaron? ¿Cuál fue la más útil? Incluyan citas con estándares adecuados.

La principal referencia utilizada fue el enunciado oficial del proyecto, ya que permitió identificar los requisitos funcionales, de diseño, usabilidad, construcción y entrega. También se utilizaron como referencia las clases proporcionadas en el proyecto shapes, especialmente Rectangle, Circle y Canvas, para comprender cómo realizar la representación gráfica del simulador.

Adicionalmente, se consultó GeeksforGeeks para aclarar conceptos de Java y reforzar temas relacionados con colecciones, clases y métodos. La referencia más útil fue el documento oficial del proyecto, porque permitió verificar de manera continua qué funcionalidades debía incluir el simulador y cuáles eran los productos requeridos para la entrega.

Referencias

Escuela Colombiana de Ingeniería Julio Garavito. (2026). Proyecto inicial Ciclo No. 1 2026-2: Desarrollo Orientado por Objetos [DOPO-POOB].

Kölling, M., & Barnes, D. J. (2000). BlueJ Shapes Example: Circle, Rectangle, Triangle and Canvas [Código fuente Java].

GeeksforGeeks. (s. f.). Java. <https://www.geeksforgeeks.org/java/java/>